

CAMPUS LIBRARY MANAGEMENT SYSTEM USING JAVA

Project Report

Submitted for the Course - Programming in Java

Submitted By:

Name: Avishi Patidar

Registration Number: 25BAI10071

Program: B.Tech / CSE in Artificial Intelligence and Machine Learning

University: VIT Bhopal University

Academic Year: 2026–27

Date of Submission: 17/09/2026

1. Introduction

The Campus Library Management System is a Java-based application developed to manage basic activities of a college library.

The system provides a simple way to manage books and student information. It also allows the user to issue and return books, maintain issue records, search for books, calculate fines for late returns, and view a library summary.

The project is developed using Core Java concepts such as classes and objects, inheritance, polymorphism, collections, exception handling, file handling, and the Java Date and Time API.

The application follows a menu-driven approach, where the user selects an option from the main menu and performs the required library operation.

The main purpose of this project is to demonstrate the practical use of Java programming concepts by developing a simple and functional library management application.

2. Problem Statement

Managing books, students, and book issue and return records manually in a college library can be time-consuming and may lead to errors.

Keeping track of available and issued books can also become difficult when records are maintained manually. Searching for a particular book and maintaining transaction records may require additional time and effort.

The Campus Library Management System is developed to provide a simple computerized solution for these basic library activities. The system helps manage book and student information, issue and return books, maintain issue records, calculate fines for late returns, and provide a summary of library information.

The project aims to make these activities more organized and easier to manage through a simple Java-based menu-driven application.

3. Functional Requirements

The functional requirements describe the main operations that the Campus Library Management System should perform.

3.1 Book Management

- The system shall allow the user to add new books.
- The system shall allow the user to view all available book records.
- The system shall allow the user to search for a book by title.
- The system shall prevent duplicate Book IDs.

3.2 Student Management

- The system shall allow the user to add student details.
- The system shall allow the user to view registered students.
- The system shall validate student information.
- The system shall prevent duplicate Student IDs.

3.3 Book Issue and Return

- The system shall allow a book to be issued to a registered student.
- The system shall check whether the book is available before issuing it.
- The system shall allow an issued book to be returned.
- The system shall update the book status after issue and return.

3.4 Fine Calculation

- The system shall calculate a fine when a book is returned after the allowed borrowing period.
- The fine shall be calculated based on the number of extra days.

3.5 Issue Records

- The system shall maintain records of book issue and return transactions.
- The system shall display issue date, return date, fine, and status.

3.6 Library Summary

- The system shall display the total number of books.
- The system shall display the total number of registered students.
- The system shall display the number of currently issued books.

3.7 File Storage and Error Handling

- The system shall save book, student, and transaction data in text files.
- The system shall display appropriate error messages for invalid input and invalid operations.

4. Non-Functional Requirements

The non-functional requirements describe the quality and performance characteristics of the Campus Library Management System.

4.1 Usability

- The system should have a simple and easy-to-understand menu.
- The user should be able to perform library operations easily.

4.2 Reliability

- The system should handle invalid operations without stopping unexpectedly.
- The system should provide appropriate messages when an operation cannot be completed.

4.3 Performance

- The system should perform basic library operations quickly.
- Searching and displaying records should be efficient for the size of the application.

4.4 Maintainability

- The project should be divided into separate classes according to their responsibilities.
- The code should be easy to understand and modify.

4.5 Error Handling

- The system should validate user input.
- The system should handle invalid numbers, duplicate IDs, missing records, and invalid library operations.

4.6 Resource Efficiency

- The application should use simple data structures and file storage.
- It should run using standard Java resources without requiring additional services.

5. System Architecture

5.1 Architecture Overview

The Campus Library Management System follows a simple layered structure. The user interacts with the system through the menu driven interface. The Main class takes input from the user and calls the required operations from the Library class. The Library class manages books, students, and issue records. Other classes are used for validation, file storage, exception handling, and generating the library summary.

5.2 System Architecture Flow

1. **Start:** The application starts and creates a Library object.
2. **Display Menu:** The system displays options such as adding books, viewing books, searching books, adding students, issuing books, returning books, and viewing reports.
3. **Accept User Input:** The user selects an option and enters the required details.
4. **Validate Input:** The system checks whether the entered information is valid.
5. **Process Request:** The Library class performs the selected operation.
6. **Update Records:** Book, student, or issue records are updated when required.
7. **Save Data:** FileManager stores the updated information in text files.
8. **Display Result:** The system shows a success message or an error message.
9. **Repeat or Exit:** The menu is displayed again until the user selects the Exit option.

5.3 Working of the System

1. The user selects an option from the main menu.
2. Main.java accepts the required input.
3. Input is checked using InputValidator.
4. The required operation is performed by Library.
5. Book, Student, and IssueRecord objects store the related information.
6. LibraryException handles invalid library operations.
7. FileManager saves the data in text files.

8. ReportManager displays the library summary.

6. System Architecture

The Campus Library Management System follows a simple modular architecture. It is a command-line-based Java application where users interact with the system through a menu-driven interface.

The system is divided into different components, such as user interaction, input validation, library management, file management, and report generation.

6.1 Architecture Components

1. User Interface Layer

- Provides a menu-driven interface through the command line.
- Accepts input from the user using the Scanner class.
- Displays messages, records, and results.

2. Validation Layer

- Validates book and student details.
- Checks IDs, names, email addresses, and text inputs.
- Handles invalid input through error messages.

3. Library Management Layer

- Manages books, students, and issue records.
- Provides features for adding, viewing, searching, issuing, and returning books.
- Checks book availability and student registration.

4. File Management Layer

- Creates the data folder.
- Saves book, student, and transaction details in text files.

- Uses file handling classes provided by Java.

5. Report Management Layer

- Displays the library summary.
- Shows the total number of books, students, issued books, and available books.

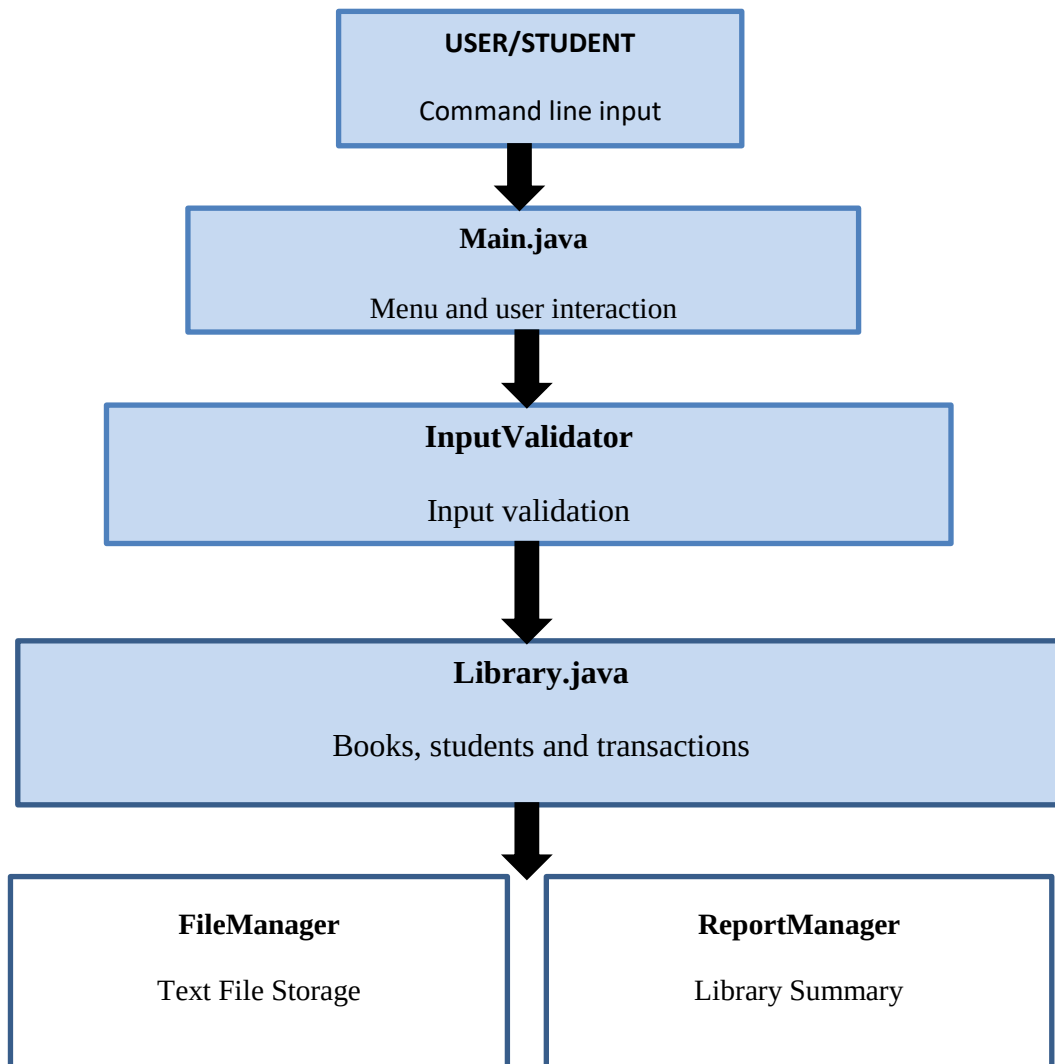
6.2 Technology Used

- Programming Language: Java
- User Interface: Command Line Interface (CLI)
- Data Storage: Text Files
- Collections: ArrayList
- Exception Handling: Custom LibraryException
- Development Environment: Java Development Kit (JDK)

6.3 Architecture Workflow

The user enters a choice through the command-line menu. The Main class receives the input and calls the relevant method of the Library class. The Library class processes the request, validates the operation, and updates the required records. FileManager saves the updated information, while ReportManager displays the library summary when requested.

The modular structure makes the application easier to understand, maintain, and extend.



7. Design Decisions and Rationale

7.1 Use of Java

Java was selected for this project because the project is developed for the Programming in Java course. Java provides object-oriented programming features, exception handling, collections, and file handling that are useful for developing this application.

7.2 Menu-Driven Interface

A menu-driven command-line interface was used to keep the application simple and easy to operate. The user can select an option from the main menu to perform the required library operation.

7.3 Use of Separate Classes

The project is divided into different classes such as Book, Student, IssueRecord, Library, and FileManager. Each class has a specific responsibility, which makes the program easier to understand and maintain.

7.4 Use of ArrayList

ArrayList is used to store books, students, and issue records. It allows the program to store multiple objects and access them easily during different operations.

7.5 Use of Exception Handling

Exception handling is used to manage invalid input and invalid library operations. A custom LibraryException class is used to display meaningful error messages for library-related errors.

7.6 Use of File Handling

Text files are used to save book, student, and transaction information. This provides simple data storage without requiring a database.

7.7 Use of Java Date and Time API

LocalDate is used to store issue and return dates. The Java Date and Time API is also used to calculate the number of days for fine calculation.

8. Implementation Details

The Campus Library Management System is implemented using Core Java. The project is divided into different classes, where each class performs a specific task. The main implementation details are given below.

8.1 Main Class

The Main class controls the overall execution of the application. It displays the main menu and takes input from the user using the Scanner class.

It uses a switch statement to perform different operations such as adding books, adding students, issuing books, returning books, searching books, and viewing the library summary.

8.2 Book Class

The Book class stores information related to a book, such as:

- Book ID
- Book title
- Author
- Category
- Availability status

The class uses private variables and getter and setter methods to implement encapsulation.

8.3 User and Student Classes

The User class stores common user information such as user ID, name, and email.

The Student class extends the User class and adds the department information. This demonstrates **inheritance**.

The displayUser() method is overridden in the Student class, demonstrating **polymorphism**.

8.4 IssueRecord Class

The IssueRecord class stores information about book transactions.

It keeps the book ID, student ID, issue date, return date, fine, and return status.

The LocalDate class is used for dates, while ChronoUnit.DAYS is used to calculate the number of days between issue and return dates.

8.5 Library Class

The Library class contains the main library operations.

It manages three ArrayList collections for:

- Books
- Students
- Issue records

The class provides operations such as adding books and students, searching books, issuing books, returning books, and displaying records.

8.6 InputValidator Class

The InputValidator class is used to check user input before performing operations.

It validates:

- Names
- Email addresses

- IDs
- Text fields

This helps prevent invalid data from being added to the system.

8.7 LibraryException Class

LibraryException is a custom exception created for handling library-related errors.

It is used for situations such as duplicate IDs, missing books or students, and trying to issue or return a book in an invalid situation.

8.8 FileManager Class

The FileManager class handles saving library information into text files.

The system saves:

- Book information in books.txt
- Student information in students.txt
- Transaction information in transactions.txt

File handling is implemented using Java's File and FileWriter classes.

8.9 ReportManager Class

The ReportManager class generates a simple library summary.

It displays:

- Total number of books
- Total number of students
- Number of currently issued books

8.10 Data Storage

The project uses simple text files for storing library data. A separate file is maintained for books, students, and transactions.

9. Screenshots:

9.1 Project Successfully Run:

```
PS D:\Java\CampusLibraryManagement\src\library> cd D:\Java\CampusLibraryManagement
>> javac -d out src\library\*.java
>> java -cp out library.Main

CAMPUS LIBRARY MANAGEMENT
1. Add Book
2. View Books
3. Search Book
4. Add Student
5. View Students
6. Issue Book
7. Return Book
8. View Issue Records
9. Library Summary
10. Exit
Enter your choice: █
```

9.2 Add Book:

```
Enter your choice: 1
Enter Book ID: 1
Enter Book Title: Java
Enter Author: Henry
Enter Category: Programming
Book added successfully.
Book data saved successfully.
```

9.3 Add Student:

```
Enter your choice: 4
Enter Student ID: 1
Enter Student Name: Avishi
Enter Student Email: avishi@gmail.com
Enter Department: AIML
Student added successfully.
Student data saved successfully.
```

9.4 Issue Book:

```
Enter your choice: 6
Enter Book ID: 1
Enter Student ID: 1
Book issued successfully.
Book data saved successfully.
Transaction data saved successfully.
```

9.5 Return Book:

```
Enter your choice: 7
Enter Book ID: 1
Book returned successfully.
Fine: ?0.0
Book data saved successfully.
Transaction data saved successfully.
```

9.6 Library Summary:

```
Enter your choice: 9

LIBRARY SUMMARY
Total Books: 1
Total Students: 1
Issued Books: 0
```